

Fast Text:

Incredibly Fast Text Classification

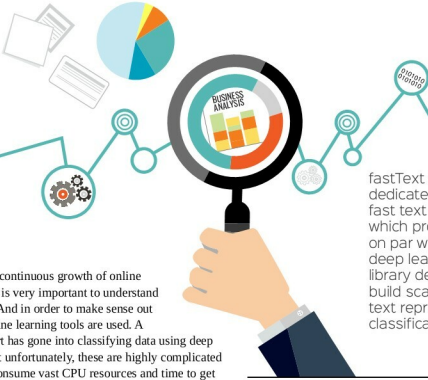
With the continuous growth of online data, it is very important to understand it too. And in order to make sense out of the data, machine learning tools are used. A great deal of effort has gone into classifying data using deep learning tools, but unfortunately, these are highly complicated procedures that consume vast CPU resources and time to get us results. fastText is the best available text classification library that can be used for blazing fast model training and for fairly accurate classification results.

Text classification is a significant task in natural language processing (NLP) as it can help us solve essential problems like filtering spam, searching the Web, page ranking, document classification, tagging and even something like sentiment analysis. Let us explore fastText in detail.

Why fastText?

fastText is an open source tool developed by the Facebook AI Research (FAIR) lab. It is a library that is dedicated to representing and classifying text in a scalable environment, and has a faster and superior performance compared to any of the other available tools. It is written in C++ but also has interfaces for other languages like Python and Node.js.

According to Facebook, "We can train fastText on more than one billion words in less than 10 minutes using a standard multi-core CPU, and classify half a million sentences among 312K classes in less than a minute." That kind of CPU-intensive classification would generally take hours to achieve using any other machine learning tool. Deep learning tools perform well on small data sets, but tend to be very slow in case of large data sets, which limits their use in production environments.



fastText is a state-of-art, dedicated tool for super-fast text classification, which provides accuracy on par with any other deep learning tool. It is a library designed to help build scalable solutions for text representation and classification.

At its core, fastText uses the 'bag of words' approach, disregarding the order of words. Also, it uses a hierarchical classifier instead of a linear one to reduce the linear time complexity to logarithmic, and to be much more efficient on large data sets with a higher category count.

Comparison and statistics

To test the fastText predictions, we used an already trained model with 9000 Web articles of more than 300 words each and eight class labels. This we looped into the Python API created using the Asyncio framework, which works in an asynchronous fashion similar to *Node.js*. We performed a test using an Apache benchmarking tool to evaluate the response time. The input was *lorem ipsum* text of about 500 lines as a single document for text classification. No caching was used in any of the modules to keep the test results sane. We performed 1000 requests, with 10 concurrent requests each time, and got the results shown in Figure 1.

The result states that the average response time was 8 milliseconds and the maximum response time was 11 milliseconds. Table 1 shows the training time required and accuracy achieved by fastText when compared to other popular deep learning tools, as per the data presented by Facebook in one of its case studies.